5. Write unit tests. Exactly what unit tests you need to write depends on the situation. You'll probably have model logic to test, and maybe a plain Ruby object managing a transaction. The important thing is that the unit tests may go beyond making the integration test work. They also have some responsibility to cover edge and error cases that integration tests wouldn't cover.
6. When you think the unit tests are done, go back to the integration test. There's a good chance it still fails because you forgot some piece of integration. No problem; that's what the integration test is there for.
7. The integration test passes. Yay!
8. Refactor the code you've written if needed.
9. If there's another significant user case in the feature, write the test for it and start over.

Let's go back to your test and see how this plays out in practice.

Making the Capybara Test Pass

Let's go through the integration-test process. One quick note: you're using factory_bot (see Chapter 6, *Adding Data to Tests*, on page 101) to create the projects and tasks with let!. You could also use fixtures, but for this go-around I've decided the factories are more readable, if slower.

The first failure is that you don't have a show method in the ProjectsController. The show method is easy enough, and probably doesn't need additional testing:

integration/01/app/controllers/projects_controller.rb

```
def show
  @project = Project.find(params[:id])
end
```

You'll also want a template. You know it's going to need a table for the tasks as well as a form to create a new task. Here's one. It's unstyled, but it's got the table and the form to create a new task:

integration/01/app/views/projects/show.html.erb

```
<h2>Project <%= @project.name %></h2>

<h3>Existing Tasks:</h3>

<table>
  <thead>
    <tr>Name</tr>
    <tr>Size</tr>
  </thead>
```